OFFICIAL START OF NOTEBOOK

Imports

In [1]:
```
import time
import os
import math
import random
import matplotlib.pyplot as plt
from collections import namedtuple, deque
from itertools import count
import copy

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from animationenv import AnimationEnvironment
```

Important classes - functions

In [2]:
```
def create_run_dir():
    if not os.path.exists(EXP_DIR):
        os.mkdir(EXP_DIR)
        print("Made the experiments dir")
    else:
        print("Experiments dir already exists!")

    run_num = len(os.listdir(EXP_DIR))

    run_dir = os.path.join(EXP_DIR, "run_{}".format(run_num))
    os.mkdir(run_dir)
    print("Created {}".format(run_dir))

    return run_dir
```

In [3]:
```
device = torch.device(
    "cuda" if torch.cuda.is_available() else
    "mps" if torch.backends.mps.is_available() else
    "cpu"
)
```

In [4]:
```
class ReplayMemory(object):
```

```
    def __init__(self, capacity):
        self.memory = deque([], maxlen = capacity)

    def push(self, state, action, next_state, reward):
        self.memory.append((state, action, next_state, reward))

    def sample(self, batch_size):
        rs = random.sample(self.memory, batch_size)
        states, actions, next_states, rewards = zip(*rs)
        return list(states), list(actions), list(next_states), list(rewards)

    def __len__(self):
        return len(self.memory)
```

In [5]:
```
class DQN(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 128)
        self.layer2 = nn.Linear(128, 128)
        self.layer3 = nn.Linear(128, 128)
        self.layer4 = nn.Linear(128, n_actions)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        x = F.relu(self.layer3(x))
        return self.layer4(x)
```

Hyper Parameters Neccessary for training:

In [6]:
```
EPS_START = 0.9
EPS_END = 0.05
EPS_DECAY = 50
BATCH_SIZE = 10

EPISODES = 500
MODEL_SAVE = 100
EPISODE_LEN = 100
EXP_DIR = "./experiments"

GAMMA = 0.99

TAU = 0.005
LR = 1e-4

start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]] #This is just the default st
```

In [7]:
```
def select_action(state, num_steps, policy_net, env):
    eps_curr = EPS_END + (EPS_START - EPS_END)*math.exp(-1*num_steps/EPS_DEC
    print("exploration rate: ", eps_curr)

    coin = random.random()

    if coin > eps_curr:
        #select the greedy action
        with torch.no_grad():
            return policy_net(state).max(1).indices.view(1,1).item(), True
    else:
        return env.sampleActionSpace(), False
```

In [8]:
```
def select_greedy_action(state, policy_net):
    with torch.no_grad():
            return policy_net(state).max(1).indices.view(1,1).item()
```

In [9]:
```
def optimize_model(policy_net, target_net, optimizer, memory):
    if len(memory) < BATCH_SIZE:
        return

    states , actions, next_states, rewards = memory.sample(BATCH_SIZE)

    non_final_mask = torch.tensor(tuple(map(lambda s: s is not None, next_st

    non_final_next_states = torch.cat([s for s in next_states if s is not No

    state_batch = torch.cat(states)
    action_batch = torch.cat(actions).unsqueeze(1)
    reward_batch = torch.cat(rewards)

    state_action_values = policy_net(state_batch).gather(1, action_batch)

    next_state_values = torch.zeros(BATCH_SIZE, device = device)

    with torch.no_grad():
        next_state_values[non_final_mask] = target_net(non_final_next_states

    expected_state_action_values = (next_state_values * GAMMA) + reward_batc

    criterion = nn.SmoothL1Loss()
    loss = criterion(state_action_values, expected_state_action_values.unsqu
    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_value_(policy_net.parameters(), 100)
    optimizer.step()
```

In [10]:
```
#Now let us implement the full training loop, this will be fun
```

```
def train(env: AnimationEnvironment):

    n_acts = env.action_space
    state, _,_ = env.get_state()
    n_obs = state.shape[1]

    policy_net = DQN(n_obs, n_acts).to(device)
    target_net = DQN(n_obs, n_acts).to(device)
    target_net.load_state_dict(policy_net.state_dict())

    optimizer = optim.AdamW(policy_net.parameters(), lr=LR, amsgrad=True)
    memory = ReplayMemory(10000)

    print("Starting training...")
    print("n_observations: {}, n_actions: {}".format(n_obs, n_acts))
    time.sleep(2)
    print("Started training...")

    rewards_per_episode = []

    for episode in range(EPISODES + 1):

        print("Starting Episode: {}".format(episode))

        env.reset()
        state, _, _ = env.get_state()

        total_reward = 0.

        for episode_iter in range(EPISODE_LEN):

            action, greedy = select_action(state, episode/4, policy_net, env
            print("action selected: {}, greedy: {}".format(action, greedy))
            next_state, reward, terminated = env.step(action)
            total_reward += reward

            print("Current grouping: {}, area score: {}, reward: {}", env.gr

            reward = torch.tensor([reward], device = device)
            action = torch.tensor([action], device = device)

            if terminated:
                next_state = None

            memory.push(state, action, next_state, reward)

            state = next_state

            optimize_model(policy_net, target_net, optimizer, memory)
```

```
            #now we have the slow update of the target network
            for target_param, policy_param in zip(target_net.parameters(), p
                target_param.data.copy_(TAU * policy_param.data + (1 - TAU)

            if terminated:
                print("Episode {} has ended at iteration {}".format(episode,
                break

        rewards_per_episode.append(total_reward)

        if episode % MODEL_SAVE == 0:
            torch.save(policy_net.state_dict(), os.path.join(RUN_DIR, 'polic
            torch.save(target_net.state_dict(), os.path.join(RUN_DIR, 'targe

    return rewards_per_episode, policy_net, target_net
```

In [11]:
```
def plot_rewards_per_episode(rewards, title="Rewards per Episode", xlabel="E
    # Create a figure and axis
    plt.figure(figsize=(10, 6))

    # Plot the rewards
    plt.plot(rewards, marker='o', linestyle='-', color='b', label='Reward pe

    # Add labels and title
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)

    # Add a grid for better readability
    plt.grid(True)

    # Add a legend
    plt.legend()

    # Show the plot
    plt.show()
```

Jab training:

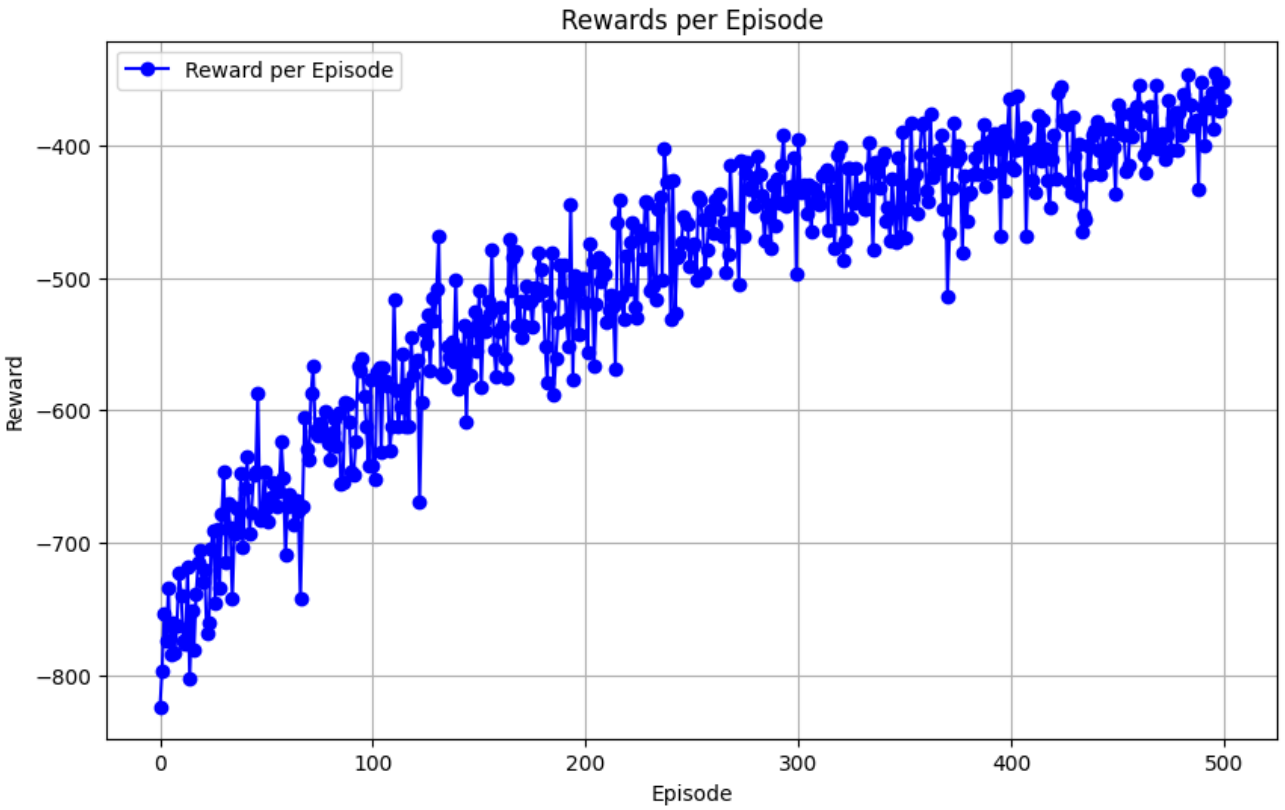
In [11]:
```
RUN_DIR = create_run_dir()
```

```
Experiments dir already exists!
Created ./experiments/run_8
```

In [12]:
```
jab_env = AnimationEnvironment("jab", start_groupings, 26, 100)
```

In [ ]:
```
jab_rewards_per_episode, pnet_1, tnet_1 = train(jab_env)
```

In [14]:
```
plot_rewards_per_episode(jab_rewards_per_episode)
```



Knee Training:

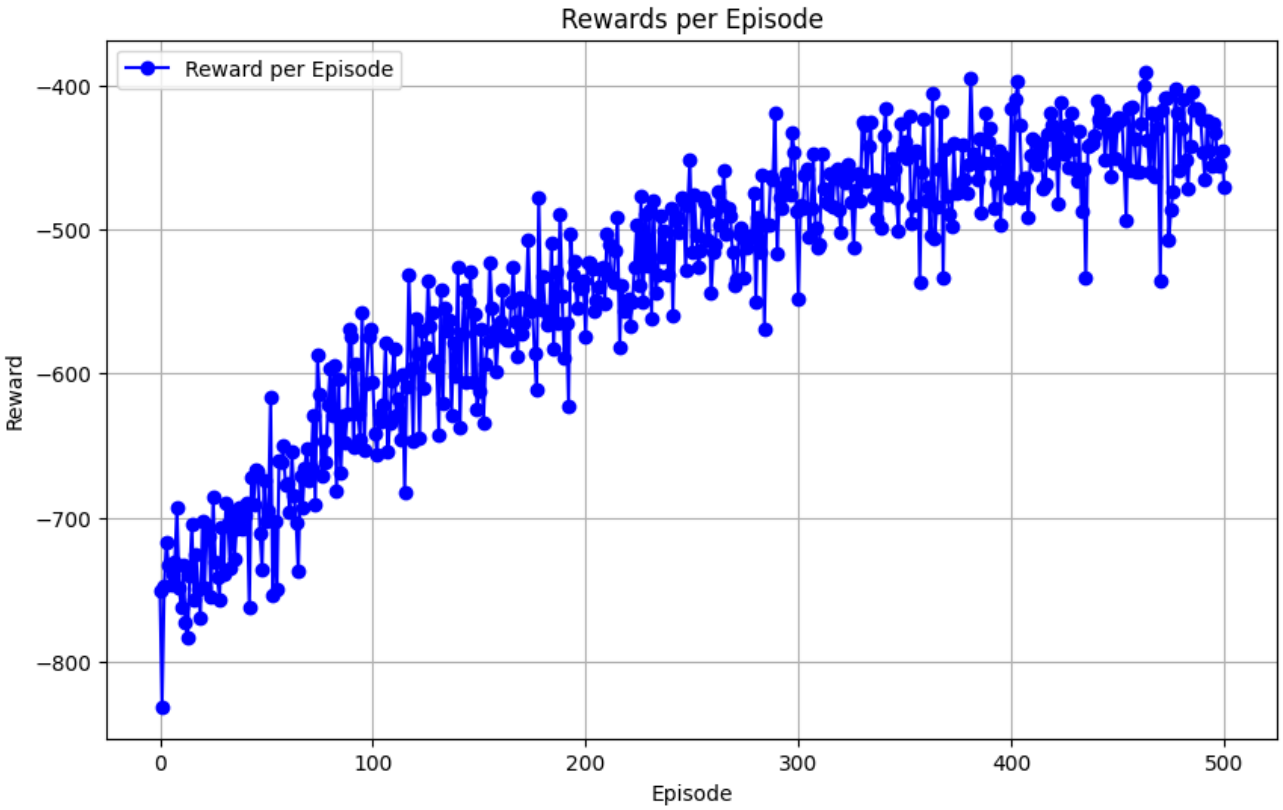
In [15]:
```
RUN_DIR = create_run_dir()
```

```
Experiments dir already exists!
Created ./experiments/run_9
```

In [16]:
```
knee_env = AnimationEnvironment("knee", start_groupings, 26, 100)
```

In [ ]:
```
knee_rewards_per_episode, pnet_2, tnet_2 = train(knee_env)
```

In [18]:
```
plot_rewards_per_episode(knee_rewards_per_episode)
```



FlipKick Training:

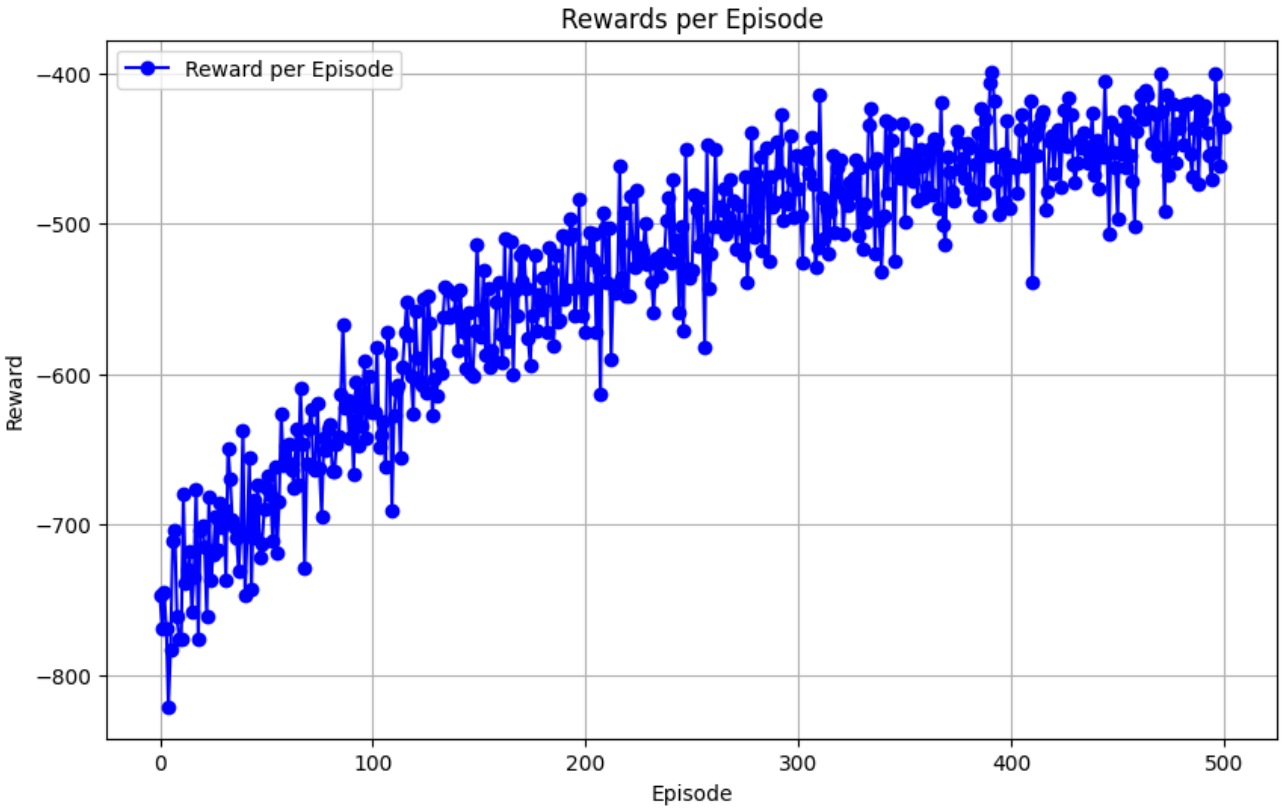
In [19]:
```
RUN_DIR = create_run_dir()
```

```
Experiments dir already exists!
Created ./experiments/run_10
```

In [20]:
```
flipkick_env = AnimationEnvironment("flipkick", start_groupings, 26, 100)
```

In [ ]:
```
flipkick_rewards_per_episode, pnet_3, tnet_3 = train(knee_env)
```

In [22]:
```
plot_rewards_per_episode(flipkick_rewards_per_episode)
```



Test the RL algorithm:

save the models:

In [26]:
```
RUN_DIR = create_run_dir()

torch.save(pnet_1.state_dict(), os.path.join(RUN_DIR, 'pjab_state_dict.pth')
torch.save(pnet_2.state_dict(), os.path.join(RUN_DIR, 'pknee_state_dict.pth'
torch.save(pnet_3.state_dict(), os.path.join(RUN_DIR, 'pflipkick_state_dict.
torch.save(tnet_1.state_dict(), os.path.join(RUN_DIR, 'tjab_state_dict.pth')
torch.save(tnet_2.state_dict(), os.path.join(RUN_DIR, 'tknee_state_dict.pth'
torch.save(tnet_3.state_dict(), os.path.join(RUN_DIR, 'tflipkick_state_dict.
```

```
Experiments dir already exists!
Created ./experiments/run_11
```

In [27]:
```
def test_rl_model(env: AnimationEnvironment, policy_net):

    print("Starting Evaluation: ")

    env.reset()
    state, _, _ = env.get_state()

    first_grouping = copy.deepcopy(env.start_groupings)
```

```
    total_reward = 0.

    for episode_iter in range(EPISODE_LEN):

        action = select_greedy_action(state, policy_net)
        print("action selected: {}".format(action))
        next_state, reward, _ = env.step(action)
        total_reward += reward
        state = next_state

        print("groupings: {}, area: {}".format(env.groupings, env.area_score

    _, last_area_score, _ = env.get_state()

    last_grouping = copy.deepcopy(env.groupings)


    return total_reward, first_grouping, last_grouping
```

In [29]:
```
def find_avg_area(env: AnimationEnvironment, g1: list[list[int]]):

    env.reset()
    env.groupings = g1
    area_sum = 0

    for i in range(EPISODE_LEN):
        _, area_score, _  = env.get_state()
        area_sum += area_score
        env.step(0)

    return area_sum/EPISODE_LEN
```

In [36]:
```
RUN_DIR = "./experiments/run_11"
```

load the models:

In [ ]:
```
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
env = AnimationEnvironment("jab", start_groupings, 26, 100)

n_acts = env.action_space
state, _,_ = env.get_state()
n_obs = state.shape[1]

policy_net = DQN(n_obs, n_acts).to(device)

policy_net_state_dict = torch.load(os.path.join(RUN_DIR, 'pjab_state_dict.pt
policy_net.load_state_dict(policy_net_state_dict)
```

```
policy_net.eval()

total_reward , first_grouping, last_grouping = test_rl_model(env, policy_net

print("First and last groupings: ", first_grouping, last_grouping)

print("Comparison: first avg area: {}, last_avg_area: {}".format(find_avg_ar
```

In [ ]:
```
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
env = AnimationEnvironment("knee", start_groupings, 26, 100)

n_acts = env.action_space
state, _,_ = env.get_state()
n_obs = state.shape[1]

policy_net = DQN(n_obs, n_acts).to(device)

policy_net_state_dict = torch.load(os.path.join(RUN_DIR, 'pknee_state_dict.p
policy_net.load_state_dict(policy_net_state_dict)

policy_net.eval()

total_reward , first_grouping, last_grouping = test_rl_model(env, policy_net

print("First and last groupings: ", first_grouping, last_grouping)

print("Comparison: first avg area: {}, last_avg_area: {}".format(find_avg_ar
```

In [ ]:
```
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
env = AnimationEnvironment("flipkick", start_groupings, 26, 100)

n_acts = env.action_space
state, _,_ = env.get_state()
n_obs = state.shape[1]

policy_net = DQN(n_obs, n_acts).to(device)

policy_net_state_dict = torch.load(os.path.join(RUN_DIR, 'pflipkick_state_di
policy_net.load_state_dict(policy_net_state_dict)

policy_net.eval()

total_reward , first_grouping, last_grouping = test_rl_model(env, policy_net

print("First and last groupings: ", first_grouping, last_grouping)

print("Comparison: first avg area: {}, last_avg_area: {}".format(find_avg_ar
```

Group Environment training:

Create a model that can train on all 3 of the models simultaneously, and have a policy network and a target network trained on all three animations simultaneously

In [19]:
```
class DQNHeavy(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQNHeavy, self).__init__()
        self.layer1 = nn.Linear(n_observations, 150)
        self.layer2 = nn.Linear(150, 150)
        self.layer3 = nn.Linear(150, 150)
        self.layer4 = nn.Linear(150, n_actions)

    def forward(self, x):
        x = F.leaky_relu(self.layer1(x), negative_slope = 0.01)
        x = F.leaky_relu(self.layer2(x), negative_slope = 0.01)
        x = F.leaky_relu(self.layer3(x), negative_slope = 0.01)
        return self.layer4(x)
```

In [20]:
```
RUN_DIR = create_run_dir()
BATCH_SIZE = 30
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
```

```
Experiments dir already exists!
Created ./experiments/run_16
```

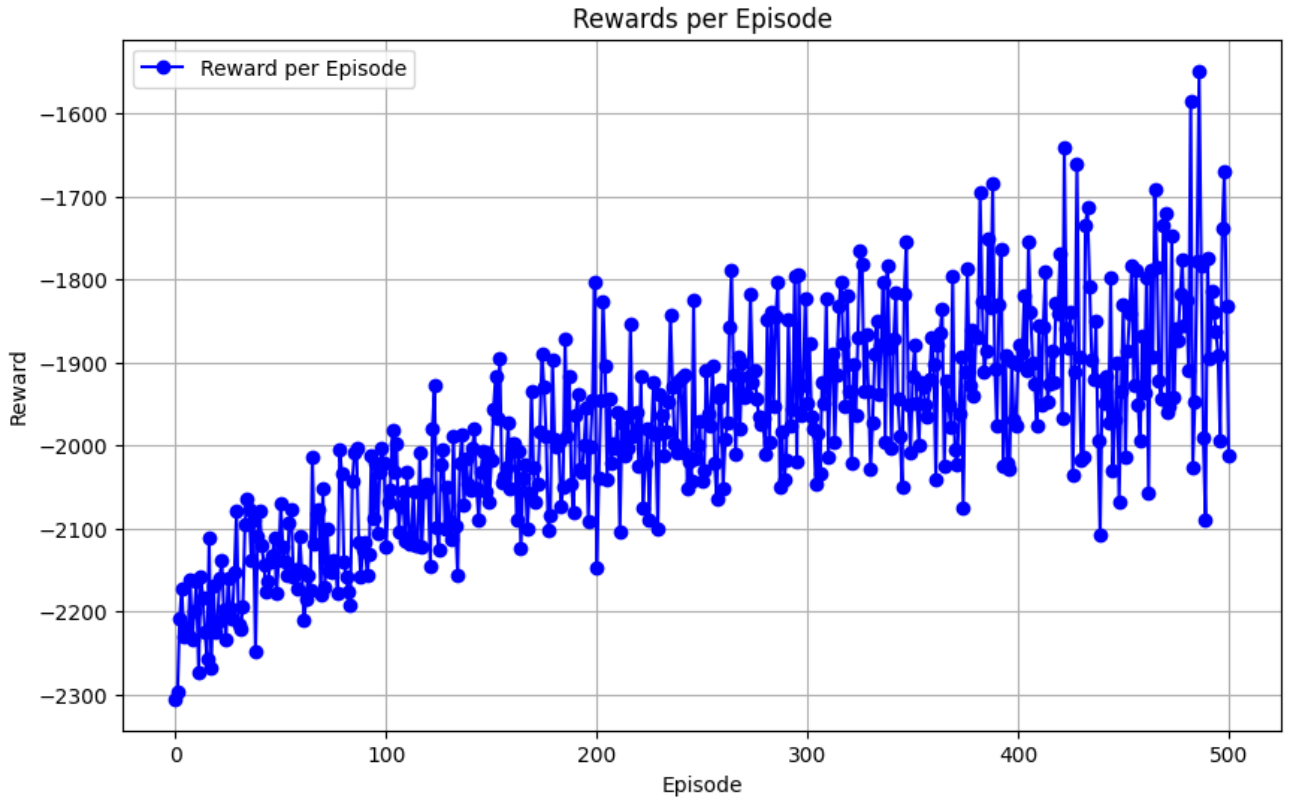
In [21]:
```
def batch_train(envs: list[AnimationEnvironment]):

    n_acts = envs[0].action_space
    state, _,_ = envs[0].get_state()
    n_obs = state.shape[1]

    policy_net = DQNHeavy(n_obs, n_acts).to(device)
    target_net = DQNHeavy(n_obs, n_acts).to(device)
    target_net.load_state_dict(policy_net.state_dict())

    optimizer = optim.AdamW(policy_net.parameters(), lr=LR, amsgrad=True)
    memory = ReplayMemory(3*10000)

    print("Starting batch training...")
    time.sleep(2)
    print("Started batch training...")

    rewards_per_episode = []

    for episode in range(EPISODES + 1):

        print("Starting Episode: {}".format(episode))
```

```
        for i in range(len(envs)):
            envs[i].reset()

        states = []

        for i in range(len(envs)):
            state, _, _ = envs[i].get_state()
            states.append(state)

        total_reward = 0.

        done = False

        for episode_iter in range(EPISODE_LEN):

            next_states = []

            for i in range(len(envs)):
                action, greedy = select_action(states[i], episode/4, policy_
                print("For env {} the action chosen was {}, greedy:{}".forma
                next_state, reward, terminated = envs[i].step(action)
                next_states.append(next_state)
                total_reward += reward

                reward = torch.tensor([reward], device = device)
                action = torch.tensor([action], device = device)

                if terminated:
                    next_state = None

                done = done or terminated

                memory.push(state, action, next_state, reward)

            states = next_states

            optimize_model(policy_net, target_net, optimizer, memory)

            #now we have the slow update of the target network
            for target_param, policy_param in zip(target_net.parameters(), p
                target_param.data.copy_(TAU * policy_param.data + (1 - TAU)

            if done:
                print("Episode {} has ended at iteration {}".format(episode,
                break

        rewards_per_episode.append(total_reward)

        if episode % MODEL_SAVE == 0:
```

```
            torch.save(policy_net.state_dict(), os.path.join(RUN_DIR, 'polic
            torch.save(target_net.state_dict(), os.path.join(RUN_DIR, 'targe

    return rewards_per_episode, policy_net, target_net
```

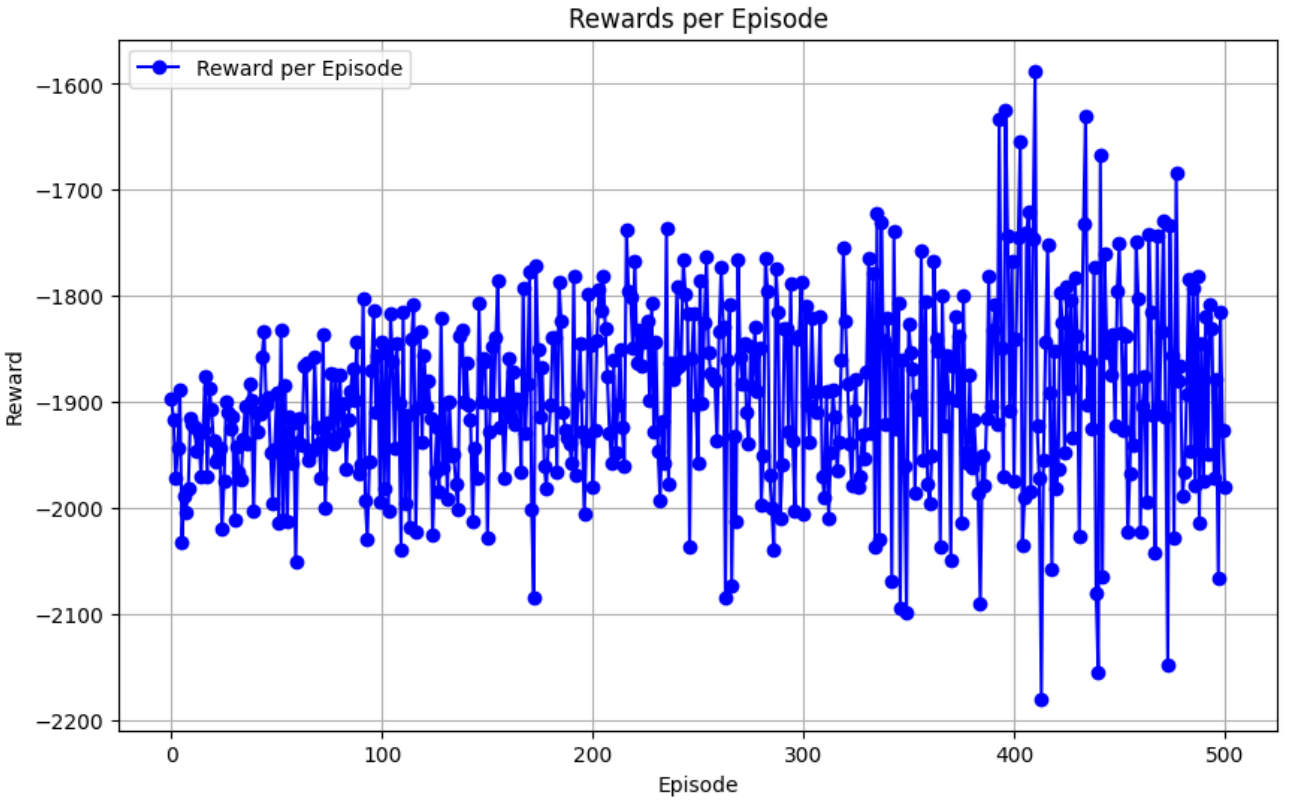
In [ ]:
```
#now we can run the batch train over here, and see the results of it, and se

#The jab knee with a tilt
envs = [AnimationEnvironment("jab", start_groupings, 26, 100), AnimationEnvi


rewards_per_episode, policy_net, target_net = batch_train(envs)
```

In [27]:
```
plot_rewards_per_episode(rewards_per_episode)
```



In [ ]:
```
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
envs = [AnimationEnvironment("jab", start_groupings, 26, 100), AnimationEnvi

for env in envs:
    env.light_action_penalty = True

rewards_per_episode, policy_net, target_net = batch_train(envs)
```

In [23]:
```
plot_rewards_per_episode(rewards_per_episode)
```



In [ ]:
```
#lets create a custom bouncing animation, and see how well the model perform
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]

env = AnimationEnvironment("bounce", start_groupings, 26, 100)

n_acts = env.action_space
state, _,_ = env.get_state()
n_obs = state.shape[1]

policy_net = DQNHeavy(n_obs, n_acts).to(device)

policy_net_state_dict = torch.load(os.path.join(RUN_DIR, 'policy_net_dict_50
policy_net.load_state_dict(policy_net_state_dict)

policy_net.eval()

total_reward , first_grouping, last_grouping = test_rl_model(env, policy_net

print("First and last groupings: ", first_grouping, last_grouping)

print("Comparison: first avg area: {}, last_avg_area: {}".format(find_avg_ar
```

Now let us test the effectiveness of K means on each of the animations!

In [30]:
```
def convert_groupings_to_list(g1dict):
```

```
    g1 = []
    for key in g1dict:

        if len(g1dict[key]) > 0:
            g1.append(g1dict[key])

    return g1
```

In [31]:
```
from kmeans import extract_data
from kmeans import k_means_analysis
from kmeans import create_groupings
from kmeans import kmeans_plot
```

In [ ]:
```
anims = ["jab", "knee", "flipkick", "bounce"]
groupings_list = []

for anim in anims:
    boneid_to_points, points, boneids = extract_data("./{}points.csv".format
    labels, cluster_centers = k_means_analysis(points, num_clusters=4)
    groupings = create_groupings(labels, boneids)
    print("Groupings for animation: {}, groupings: {}".format(anim, grouping
    groupings_list.append(convert_groupings_to_list(groupings))

print(groupings_list)
```

In [ ]:
```
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
envs = [AnimationEnvironment("jab", start_groupings, 26, 100), AnimationEnvi

avg_areas = []

for i in range(len(envs)):
    avg_areas.append(find_avg_area(envs[i], groupings_list[i]))
```

In [35]:
```
print(avg_areas)
```

```
[2.820812574939712, 4.006749392181743, 3.3821240672971142]
```

Let us repeat the analysis for Gaussian Mixture Clustering

In [36]:
```
from kmeans import extract_data
from kmeans import guassian_mixture_clustering
from kmeans import create_groupings
from kmeans import gaussian_mixture_plot
```

In [37]:
```
anims = ["jab", "knee", "flipkick", "bounce"]
groupings_list = []

for anim in anims:
```

```
    boneid_to_points, points, boneids = extract_data("./{}points.csv".format
    points, labels, gmm = guassian_mixture_clustering(points, num_clusters =
    groupings = create_groupings(labels, boneids)
    print("Groupings for animation: {}, groupings: {}".format(anim, grouping
    groupings_list.append(convert_groupings_to_list(groupings))

print(groupings_list)
```

```
(260, 10)
Groupings for animation: jab, groupings: {0: [0, 5, 6, 7, 8, 9], 1: [3, 4],
2: [1], 3: [2]}
(260, 10)
Groupings for animation: knee, groupings: {0: [3, 7, 8], 1: [5, 6, 9], 2: [
4], 3: [0, 1, 2]}
(260, 10)
Groupings for animation: flipkick, groupings: {0: [5, 6, 9], 1: [4], 2: [1,
2, 3], 3: [0, 7, 8]}
(260, 10)
Groupings for animation: bounce, groupings: {0: [1, 2, 3, 6, 8], 1: [4], 2:
[0, 5, 7], 3: [9]}
[[[0, 5, 6, 7, 8, 9], [3, 4], [1], [2]], [[3, 7, 8], [5, 6, 9], [4], [0, 1,
2]], [[5, 6, 9], [4], [1, 2, 3], [0, 7, 8]], [[1, 2, 3, 6, 8], [4], [0, 5,
7], [9]]]
```

In [ ]:
```
start_groupings = [[0,1,2,3],[4,5],[6,7],[8,9]]
envs = [AnimationEnvironment("jab", start_groupings, 26, 100), AnimationEnvi

avg_areas = []

for i in range(len(envs)):
    avg_areas.append(find_avg_area(envs[i], groupings_list[i]))
```

In [39]:
```
print(avg_areas)
```

```
[2.7364309376082696, 3.760880114783669, 3.445560049628071]
```